มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ
KING MONGKUT'S UNIVERSITY OF TECHNOLOGY NORTH BANGKOK

BUSINESS INTELLIGENCE

Chapter 12

Data Preprocessing in Python

Asst.Prof. Sakchai Tangprasert (Ph.D. in IT)
Sakchai.t@sci.kmutnb.ac.th

Analytics and Big Data for Accountants
By Jim Lindell
© 2017 Association of International Certified Professional Accountants, Inc.

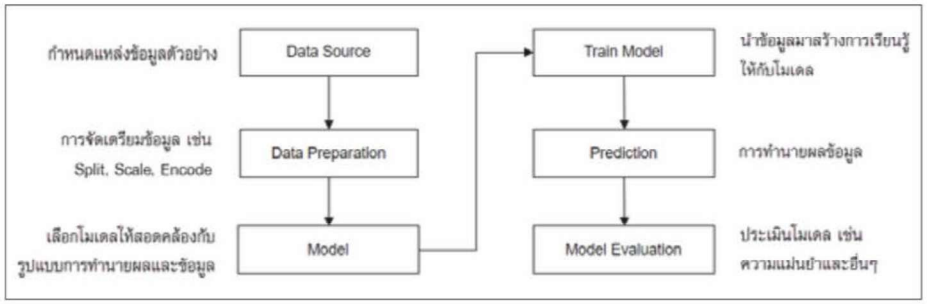
1. การจัดเตรียมข้อมูลสำหรับ Machine Learning

- องค์ประกอบที่สำคัญในลำดับแรกๆ ของ Machine Learning ก็คือข้อมูล เพราะมันคือ สิ่งจำเป็นสำหรับการ Train Model อย่างไรก็ตาม ข้อมูลที่จะนำไปใช้กับ Machine Learning ต้องอยู่ในรูปแบบที่ถูกต้องหรือสอดคล้องกับโมเดล แต่ข้อมูลที่ได้มาจากแหล่งต่างๆ ทั้งการกำหนดขึ้นมาเองหรือจากไฟล์ ก่อนจะนำข้อมูลไปใช้งานก็อาจต้องเพิ่มขั้นตอนการจัดเตรียมในลักษณะต่างๆ การคัดเลือกคอลัมน์ การปรับเปลี่ยนสเกล การเข้ารหัสรวมถึงการแบ่งเป็นชุดสำหรับ Training/Testing เป็นต้น

2. การใช้ไลบรารี Scikit-Learn สำหรับ Machine Learning

- Scikit-Learn หรือเรียกตามชื่อโมดูลเป็น SkLearn คือไลบรารีสำหรับ Machine Learning ที่ได้รับความนิยมสูงสุด ซึ่งสร้างโดยทีมนักพัฒนาของ Google เพื่อใช้งานร่วมกับโลบรารีพื้นฐานที่มีอยู่แล้วของ Data Science อย่างเช่น NumPy รวมถึง Pandas ทั้งนี้หากติดตั้งชุดแพ็กเกจ Anaconda ก็จะมี Scikit-Learn รวมอยู่ด้วย จึงสามารถใช้งานได้ทันที
- ไลบรารี Scikit-Learn ประกอบด้วยโมดูล คลาส และฟังก์ชันจำนวนมาก ซึ่งต้องเลือกอิมพอร์ตให้สอดคล้องกับวัตถุประสงค์ของการใช้งาน โดยชื่อโมดลจะเริ่มต้นด้วยคำว่า sklearn ทั้งหมด ทั้งนี้วิธีการทำนายผลด้วย Scikit-Learn ของแต่ละโมเดลส่วนใหญ่จะใช้หลักการที่คล้ายคลึงกัน ซึ่งสามารถสรุปลำดับขั้นตอนที่สำคัญได้ดังภาพ

2. การใช้ไลบรารี Scikit-Learn สำหรับ Machine Learning (ต่อ)



2. การใช้ไลบรารี Scikit-Learn สำหรับ Machine Learning (ต่อ)

- สำหรับแนวทางการเขียนโค้ดของขั้นตอนหลัก ๆ จะเป็นดังนี้

```
#อิมพอร์ตโมดูลทั้งหมดที่จำเป็นต้องใช้งาน
from sklearn....._ i
import ClassName
from sklearn.__ import function_name
#กำหนดแหล่งข้อมูลตัวอย่าง ด้วยวิธีใดวิธีหนึ่ง
data = pd.read_csv(...)
#บางกรณีอาจต้องเพิ่มขั้นตอนการจัดเตรียมข้อมูลเพิ่มเติม
#เช่น การเข้ารหัสสตริงเป็นตัวเลข การแบ่งข้อมูล การปรับสเกล เป็นต้น
encoder = LabelEncoder()
x_train, x_test, y_train, y_test = train_test_split(...)
scaler = StandardScaler()
#เลือกโมเดลที่จะใช้ในการทำนายผล
```


2. การใช้ไลบรารี Scikit-Learn สำหรับ Machine Learning (ต่อ)

```
model = ModelName(...)
#นำข้อมูลมากำหนดให้แก่โมเดล เพื่อสร้างการเรียนรู้
#โดยเรียกขั้นตอนนี้ว่าการ Train Model
#ซึ่งแต่ละโมเดลจะมีเมธอดที่ชื่อ fit( ) สำหรับการ Train
model.fit(...)
#หากต้องการทำนายผล ให้ระบุข้อมูลที่จะทำนาย
#แล้วกำหนดให้แก่เมธอด predict ( ) ของโมเดล
data_for_prediction = ...
predict = model.predict(data_for_prediction)
#อาจตรวจสอบความถูกต้องการทำนายผล ซึ่งมีหลากหลายรูปแบบให้เลือก เช่น
score = model.score(...)
```


3. ลักษณะข้อมูลสำหรับ Machine Learning

- ปัจจัยที่ใช้ในการแบ่งประเภทของ Machine Learning ก็คือชุดข้อมูลที่นำมาใช้ร่วมกับโมเดล (Train Model) ดังนี้
- Supervised Learning กรณีนี้ข้อมูลที่นำมาใช้จะต้องมีคอลัมน์ที่เป็นเป้าหมาย (Targe) หรือผลลัพธ์ซึ่งต้องเป็นแบบใดแบบหนึ่งระหว่าง Classification (สามารถแบ่งกลุ่มผลลัพธ์ได้ เช่น ฝนตกหรือไม่ตก) หรือแบบ Regression (ผลลัพธ์เป็นตัวเลขที่มีค่าต่อเนื่องและไม่สามารถแบ่งกลุ่มได้ เช่นปริมาณฝนที่ตก) ดังตารางการเปรียบเทียบในภาพถัดไป ซึ่งด้านซ้ายเป็นข้อมูลเกี่ยวกับสภาพอากาศและผลลัพธ์เป็นการตัดสินใจว่าจะเล่นเทนนิสหรือไม่ (Classification) ส่วนด้านขวาเป็นข้อมูลเกี่ยวกับรถยนต์และผลลัพธ์เป็นปริมาณการปล่อยก๊าซคาร์บอนไดออกไซด์ (Regression)

3. ลักษณะข้อมูลสำหรับ Machine Learning (ต่อ)

Outlook	Temp	Humidity	Wind	Play
Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
Overcast	Hot	High	Weak	Yes
Rain	Mild	High	Weak	Yes
Rain	Cool	Normal	Weak	Yes
Rain	Cool	Normal	Strong	No
Overcast	Cool	Normal	Strong	Yes
Sunny	Mild	High	Weak	No
Sunny	Cool	Normal	Weak	Yes
Rain	Mild	Normal	Weak	Yes
Sunny	Mild	Normal	Strong	Yes
Overcast	Mild	High	Strong	Yes
Overcast	Hot	Normal	Weak	Yes
Rain	Mild	High	Strong	No

Brand	Model	Volume	Weight	CO2
Toyota	Aygo	1000	790	99
Mitsubishi	Space Star	1200	1160	95
Skoda	Citigo	1000	929	95
Fiat	500	900	865	90
Mini	Cooper	1500	1140	105
VW	Up!	1000	929	105
Skoda	Fabia	1400	1109	90
Mercedes	A-Class	1500	1365	92
Ford	Fiesta	1500	1112	98
Audi	A1	1600	1150	99
Hyundai	I20	1100	980	99
Suzuki	Swift	1300	990	101
Ford	Fiesta	1000	1112	99
Toyota	Aygo	1000	790	99

3. ลักษณะข้อมูลสำหรับ Machine Learning (ต่อ)

- Unsupervised Learning กรณีนี้ข้อมูลที่นำมาเทรน Train Model จะไม่มีคอลัมน์ที่เป็นเป้าหมายหรือผลลัพธ์ในลักษณะดังนี้

ID	Age	Weight	Height
1	50	65	168
2	40	70	172
3	35	59	170
4	45	64	165
5	55	70	174
6	33	63	157
7	46	64	169
8	53	75	177
9	37	73	160
10	41	61	159

3. ลักษณะข้อมูลสำหรับ Machine Learning (ต่อ)

- โดยส่วนใหญ่แล้ว โมเดลต่างๆ ของ Machine Learning จัดอยู่ในประเภท Supervised Learning จะแบ่งออกเป็น 2 ส่วนคือ
- ส่วน Features หรือตัวแปรอิสระ (Independence Variable) ซึ่งมักแทนด้วยตัว x คือกลุ่มคอลัมน์ที่จะนำไปใช้คำนวณหรือ Train Model เพื่อการทำนายผล โดยคอลัมน์ในกลุ่มนี้จะต้องเป็นตัวเลข หากเป็นสตริงต้องเข้ารหัสหรือแปลงให้เป็นตัวเลขก่อน และจะมีกี่คอลัมน์ก็ได้
- ส่วน Target หรือ (Class) Label หรือตัวแปรตาม (Dependence Variable) ซึ่งมักแทนด้วยตัว y คือกลุ่มคอลัมน์ที่เป็นเป้าหมายของการทำนายผล โดยคอลัมน์ในกลุ่มนี้จะเป็นตัวเลขหรือสตริงก็ได้โดยปกติจะมีเพียงคอลัมน์เดียว

3. ลักษณะข้อมูลสำหรับ Machine Learning (ต่อ)

Features | Target

Give Birth	Can Fly	Live in Water	Has Legs	Species
Yes	No	No	Yes	Mammals
No	No	No	No	Retiles
Yes	No	Yes	No	Fishes
No	No	Sometimes	Yes	Amphibians
No	Yes	No	Yes	Birds
...	...	...	...	...

Samples

3. ลักษณะข้อมูลสำหรับ Machine Learning (ต่อ)

- หากใช้ Scikit-Learn จะต้องระบุลงไปด้วยว่ามีคอลัมน์ใดบ้างที่เป็น Feature ซึ่งจะนำไป Train Model และคอลัมน์ใดที่เป็น Target อย่างไรก็ตาม คอลัมน์ที่จะเลือกเป็น Feature นั้น ควรมีความสัมพันธ์กับคอลัมน์ ที่เป็น Target หรือกล่าวได้ว่า คอลัมน์ที่เป็น Feature ต้องมีผลต่อการเปลี่ยนแปลงในคอลัมน์ที่เป็น Target
- ดังนั้น ไม่จำเป็นต้องเลือกคอลัมน์ทั้งหมดที่มีมาเป็น Feature เสมอไป แต่อาจเลือกเพียงบางคอลัมน์ที่มีผลต่อ Target ก็ได้ ดังตัวอย่างต่อไปนี้

3. ลักษณะข้อมูลสำหรับ Machine Learning (ต่อ)

- ซึ่งแสดงข้อมูลเกี่ยวกับรถยนต์บางรุ่นเทียบกับขนาดเครื่องยนต์น้ำหนักและปริมาณการปล่อยก๊าซคาร์บอนไดออกไซด์

car	color	volume	weight	co2
Honda Civic	Red	1600	1252	94
Suzuki Swift	White	1300	990	101
Mazda 3	Black	2200	1280	104
Benz CLA	Black	1500	1465	102
Mini Cooper	Red	1500	1140	105
...	...	...	...	...

3. ลักษณะข้อมูลสำหรับ Machine Learning (ต่อ)

- จากข้อมูลดังกล่าวนี้ คงพอคาดคะเนได้ว่า ยี่ห้อ/รุ่น และสีของรถ ไม่น่าจะเกี่ยวกับปริมาณการปล่อยก๊าซคาร์บอนไดออกไซด์โดยตรง แต่สิ่งที่มีผลน่าจะเป็นขนาดเครื่องยนต์และน้ำหนักรถมากว่า ดังนั้นนั้นควรเลือกแค่คอลัมน์ volume และ weight เป็น Feature เช่น

```
#features = df[['volume', 'weight']]
#features = df.iloc[:, 2:4]                    #เลือกทุกแถว คอลัมน์ลำดับที่ 2 - 3
#ยังมีวิธีการอื่นๆ อีกหลายแบบ
target = df['co2']
```


4. การกำหนดแหล่งข้อมูลสำหรับ Machine Learning

- ข้อมูลที่จะนำมาใช้กับ Machine Learning หากเป็นการใช้งานจริง ก็มักจัดเก็บไว้ในไฟล์หรือไม่ก็ฐานข้อมูลแต่หากเป็นการทดสอบในเบื้องต้น หากไม่มีข้อมูลที่ใกล้เคียงกับรูปแบบที่ต้องการ ก็อาจสมมติขึ้นมาเอง หรือ สร้างโดยใช้ฟังก์ชันของ Scikit-Learn ก็ได้ แต่ก็มีบางกรณีที่ต้องอาศัยการสร้างข้อมูลขึ้นมาเอง เพื่อให้ตรงตามลักษณะของโมเดลที่ต้องการทดสอบ ซึ่งทางเลือกในการกำหนดแหล่งข้อมูลสำหรับ Machine Learning มีดังนี้

5. การกำหนดข้อมูลขึ้นมาเอง

- ข้อมูลที่จะใช้กับ Machine Learning ต้องมีลักษณะเป็นตารางที่ประกอบด้วยแถวและคอลัมน์ ซึ่งหากต้องการทดสอบแบบง่ายๆ ก็อาจสมมติข้อมูลขึ้นมาเองในแบบลิสต์หรือดิกชันนารี และอาจนำไปแปลงเป็น DataFrame เพื่อให้จัดการได้ง่ายขึ้น เช่น หากข้อมูลที่ต้องการเป็นดังตารางถัดไป ก็สามารถเขียนเป็น Code ของ Python ได้ดังนี้

5. การกำหนดข้อมูลขึ้นมาเอง (ต่อ)

```
import pandas as pd

data = [[10, 20, 'A'],   #แถวที่ 1
        [5, 15, 'B'],    #แถวที่ 2
        [9, 13, 'B'],    #แถวที่ 3
        [8, 16, 'A']]    #แถวที่ 4

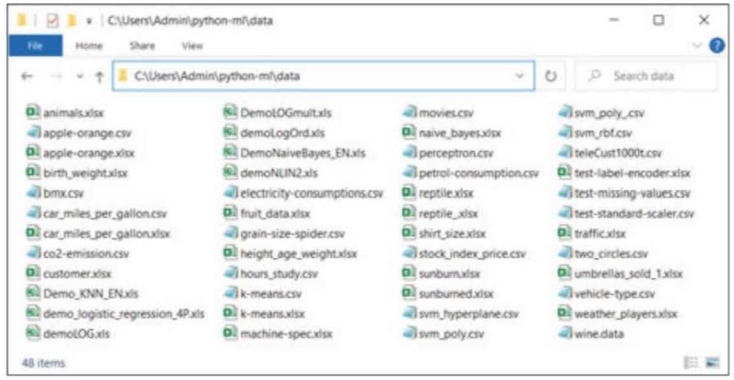
df = pd.DataFrame(data, columns=['x1', 'x2', 'y'])
display(df)
#ถ้าจะนำไปใช้กับ Machine Learning ก็เลือกคอลัมน์ที่เป็น Features และ Target
# features = df[['x1', 'x2']]
# target = df['y']
```

x1	x2	y
10	20	A
5	15	B
9	13	B
8	16	A



6. การอ่านข้อมูลจากไฟล์ CSV และ Excel

- ข้อมูลที่นำมาใช้กับ Machine Learning มักจัดเก็บอยู่ในรูปแบบของไฟล์ CSV หรือ Excel เป็นส่วนใหญ่ ซึ่งการนำมาใช้งานก็แค่อ่านเนื้อหาจากไฟล์ด้วยเมธอดของ Pandas นั่นคือ read_csv() หรือไม่ก็ read_excel() ตามชนิดของไฟล์ ซึ่งข้อมูลจะถูกจัดเก็บในออบเจ็กต์



6. การอ่านข้อมูลจากไฟล์ CSV และ Excel (ต่อ)

```
df_csv = pd.read_csv(r'dataitest.csv')
df_excel = pd.read_excel(r'datalexample.xlsx' )
display (df_csv.head(3))
...
```

- หลังจากนั้น ก็นำ DataFrame ไปใช้ร่วมกับ Machine Learning ได้เลย อย่างไรก็ตาม การอ่านมูลด้วยเมธอด read_csv() หรือ read_excel() สามารถกำหนดออปชันเพิ่มเติมได้ดังนี้

6. การอ่านข้อมูลจากไฟล์ CSV และ Excel (ต่อ)

filepath หรือ URL	ระบุตำแหน่งไฟล์ในแบบสตริง เช่น read_csv(r'c:\data.csv')
header	กรณีที่ไฟล์ไม่ได้ระบุชื่อคอลัมน์เอาไว้ ให้เราระบุค่าเป็น None ซึ่งชื่อคอลัมน์จะเป็น 0, 1, 2, 3, ...
prefix	กรณีที่ไฟล์ไม่ได้ระบุชื่อคอลัมน์เอาไว้ แล้วเราต้องการระบุคำนำหน้าหมายเลขคอลัมน์ เช่น ถ้ากำหนด prefix='col' ชื่อคอลัมน์จะเป็น col0, col1, col2, ...
names	กรณีที่ไฟล์ไม่ได้ระบุชื่อคอลัมน์เอาไว้ ถ้าเราต้องการกำหนดคอลัมน์ก็ระบุในแบบลิสต์ และควรใช้ร่วมกับ header=None เช่น names=['col1', 'col2', 'col3', ...]
index_col	หากต้องการให้คอลัมน์ใดคอลัมน์หนึ่งเป็น Index เช่น index_col='col2' หรือหากไม่ต้องการให้ Pandas นำคอลัมน์แรกมาเป็น Index ก็กำหนดค่าเป็น index_col=False
usecols	อ่านค่าจากเพียงบางคอลัมน์มาใช้งาน เช่น usecols=['col1', 'col3', 'col6']
thousands	ระบุอักขระที่ใช้คั่นหลักพัน (ปกติคือ ,)

6. การอ่านข้อมูลจากไฟล์ CSV และ Excel (ต่อ)

comment	ระบุสตริงที่ใช้เขียนคำอธิบายที่ตอนต้นบรรทัด เช่น ถ้ากำหนดเป็น comment='#' หมายถึง บรรทัดที่ขึ้นต้นด้วย # จะไม่ถือเป็นข้อมูล (ไม่อ่านข้อมูลบรรทัดนั้น)
dayfirst	กรณีข้อมูลชนิดวันเดือนปี ถ้าต้องการให้ค่าแรกหมายถึงวันที่ (เช่น DD/MM/YYYY) ก็กำหนดค่าเป็น True (ค่าปกติคือ False)
decimal	อักขระที่ใช้เป็นจุดทศนิยม (ปกติคือ .) แต่บางประเทศอาจใช้ , เช่นกำหนดเป็น decimal=',' เครื่องหมาย , จะถูกแปลงเป็น . (12,34 => 12.34)
sep หรือ delimeter	สำหรับกรณีที่ใช้อักขระหรือสัญลักษณ์อย่างอื่นในการแบ่งคอลัมน์ข้อมูล (ตามปกติจะเป็น ,) เช่น sep='#'
nrows	จำนวนช่วงแถวเริ่มแรกที่ต้องการอ่านข้อมูล เช่น pd.read_csv('data.csv', nrows=3) ผลที่ได้คือ จะอ่านเฉพาะข้อมูล 3 แถวแรก
skiprows	ตัดแถวช่วงเริ่มแรกออก แล้วนำมาเฉพาะแถวที่เหลือ เช่น pd.read_csv('data.csv', skiprows=2) ผลที่ได้คือ ตัด 2 แถวแรกออก
skipfooter	ตัดแถวช่วงท้ายออก แล้วนำมาเฉพาะแถวก่อนนี้ แต่ควรใช้ร่วมกับอาร์กิวเมนต์ engine='python' เช่น pd.read_csv('data.csv', skipfooter=2, engine='python') ผลที่ได้คือ ตัด 2 แถวสุดท้ายออก

7. การสร้างข้อมูลตัวอย่างด้วย make_classification

- กรณีที่ต้องการใช้ข้อมูลตัวอย่างแบบง่ายๆ แต่มีจำนวนมาก สำหรับการทดสอบในเบื้องต้น ก็สามารถใช้ฟังก์ชันของ Scikit-Learn เพื่อสร้างข้อมูลย่างขึ้นมาใช้งานเองได้อีกทางหนึ่ง ซึ่งหากเป็นการทำนายผลแบบ Classification ก็มีฟังก์ชันที่เกี่ยวข้องโดยตรงคือ make_classification() ซึ่งมีพารามิเตอร์ที่สำคัญดังนี้
- ตัวเลขในวงเล็บคือ ค่าดีฟอลต์

make_classification(
n_samples=จำนวนรายการ (100),
n_features=จำนวนคอลัมน์ที่เป็น Feature (100),
n_classes=จำนวนกลุ่ม (Class) หรือ Labe1 ของคอลัมน์ที่เป็น Target (2),
random_state=กำหนดเป็นจำนวนเต็มตั้งแต่ 0 ขึ้นไป เพื่อให้ได้ข้อมูลชุดเดิมตลอด

7. การสร้างข้อมูลตัวอย่างด้วย make_classification (ต่อ)

- n_features จะเกี่ยวข้องกับพารามิเตอร์อื่นๆ อีก 3 ตัวคือ
- ตัวเลขที่ระบุคือค่าดีฟอลต์
  - n_informative=2
  - n_redundant=2
  - n_repeated=0

- ซึ่งข้อกำหนดคือ

n_features >= n_informative + n_redundant + n_repeated

- เช่น ถ้าใช้ตามค่าดีฟอลต์ ผลบวกคือ

n_informative + n_redundant + n_repeated = 2 2+2+ 0 = 4

แสดงว่า n_features ต้องมีค่าตั้งแต่ 4 ขึ้นไป

7. การสร้างข้อมูลตัวอย่างด้วย make_classification (ต่อ)

- n_classes จะขึ้นอยู่กับพารามิเตอร์อีก 2 ตัวคือ
- ตัวเลขที่ระบุคือค่าดีฟอลต์
  - n_informative=2
  - n_clusters_per_class=2

- ซึ่งข้อกำหนดคือ
- ** คือการยกกำลัง

n_classes <= (2 ** n_informative) / n_clusters_per_class

- เช่น ถ้าใช้ตามค่าดีฟอลต์ ผลที่ได้คือ (2 * 2) / 2 = 2 แสดงว่า n classes จะมีค่าไม่เกิน 2 แต่ถ้าต้องการจำนวนคลาสมากขึ้นก็อาจเปลี่ยนแปลงค่าของ 2 พารามิเตอร์ที่เกี่ยวข้อง โดยค่าของ n_informative จะมีผลต่อค่าของ n_features ด้วย

7. การสร้างข้อมูลตัวอย่างด้วย make_classification (ต่อ)

- random_state การกำหนดตัวเลขที่ต่างกันจะได้ชุดข้อมูลที่แตกต่างกันออกไปด้วย แต่หากเป็นเลขตัวเดิมก็จะได้ข้อมูลชุดเดียวกันไปตลอด
- ฟังก์ชัน make_classification() อยู่ในโมดูล sklearn.datasets ซึ่งต้องอิมพอร์ตก่อนการใช้งาน และผลลัพธ์ที่ได้จาก make_classification() จะเป็นอาร์เรย์ของ NumPy ในแบบ 2 มิติ ทั้งนี้ สามารถนำไปใช้กับโมเดลของ Machine Learning ได้เลย หรือจะแปลงเป็น DataFrame ก่อน เพื่อให้จัดการได้ง่ายขึ้นดังต่อไปนี้

7. การสร้างข้อมูลตัวอย่างด้วย make_classification (ต่อ)

```
from sklearn.datasets import make_classification
import pandas as pd

x, y = make_classification(n_samples=100,
                           n_features=6, n_informative=3,
                           n_redundant=2, n_repeated=0,
                           n_clusters_per_class=2,
                           n_classes=3,
                           random_state=0)

#ถ้าต้องการแปลงเป็น DataFrame
df = pd.DataFrame(x, columns=['x1', 'x2', 'x3', 'x4', 'x5', 'x6'])
df['y'] = y

with pd.option_context('display.max_rows', 6): display(df)
```



7. การสร้างข้อมูลตัวอย่างด้วย make_classification (ต่อ)

	x1	x2	x3	x4	x5	x6	y
0	-1.883154	0.806082	-1.142918	-0.512257	-0.362460	-0.159945	2
1	0.689837	1.068260	0.006068	-0.679698	0.394167	-1.191951	1
2	2.862320	-0.967497	0.524888	-0.668869	0.184036	-0.766462	1
...	...	...	...	...	...	...	...
97	1.997166	0.856933	-0.898758	-1.064800	-1.194870	0.619902	0
98	1.997506	-0.802995	2.274575	2.340079	0.161256	2.333207	0
99	-1.941414	-0.335021	2.773086	2.975389	2.074036	0.186306	2

100 rows × 7 columns

8. การสร้างข้อมูลตัวอย่างด้วย make_regression

- หากต้องการทำนายผลแบบ Regression ผลลัพธ์เป็นตัวเลขแบบค่าต่อเนื่อง ใน Scikit-Learn ก็มีฟังก์ชัน make_regression() เพื่อสร้างข้อมูลตัวอย่างสำหรับกรณีนี้โดยตรง ซึ่งพารามิเตอร์ที่สำคัญมีดังมีดังนี้
- ตัวเลขในวงเล็บคือ ค่าดีฟอลต์

make_regression(
n_samples=จำนวนรายการ (100),
n_features=จำนวนคอลัมน์ที่เป็น Feature (100),
random_state=กำหนดเป็นจำนวนเต็มตั้งแต่ 0 ขึ้นไป เพื่อให้ได้ข้อมูลชุดเดิมตลอด

8. การสร้างข้อมูลตัวอย่างด้วย make_regression (ต่อ)

- ฟังก์ชัน make_regression ไม่มีข้อกำหนดที่ยุ่งยากเหมือนกับ make_classification ตัวอย่างการใช้งาน เช่น

```
from sklearn.datasets import make_regression
import pandas as pd

x, y = make_regression(n_samples=100, n_features=5, random_state=0)

#ถ้าต้องการแปลงเป็น DataFrame
df = pd.DataFrame(x, columns=['x1', 'x2', 'x3', 'x4', 'x5'])
df['y'] = y

with pd.option_context('display.max_rows', 10): display(df)
```



8. การสร้างข้อมูลตัวอย่างด้วย make_regression (ต่อ)

	x1	x2	x3	x4	x5	y
0	-0.993124	0.280442	0.049495	-0.249459	0.841631	-69.645151
1	-2.772593	0.399046	-0.652409	0.390093	1.955912	-144.809532
2	1.029439	0.771406	0.862596	-0.424318	-0.908763	71.384309
3	1.077744	-0.133702	-0.384880	-0.730678	-1.126826	-15.383890
4	-0.539455	-2.016407	1.738873	-0.709728	-0.275671	-197.139359
...	...	...	...	...	...	...
95	0.317218	0.910179	-0.944446	-0.466419	0.786328	54.946162
96	-1.347759	1.883151	-1.173123	0.969397	-1.270485	-30.508671
97	-0.413619	1.943621	1.480515	1.922942	-0.747455	198.820567
98	-0.349943	-1.029935	2.696224	1.298022	1.100284	108.140123
99	1.494485	0.781198	0.676908	0.426259	-2.069985	138.929501

100 rows × 6 columns

9. การสร้างข้อมูลตัวอย่างด้วย make_blobs

- ในบางโมเดล ต้องใช้ข้อมูลตัวอย่างที่มีลักษณะการแบ่งกลุ่ม (Cluster) เช่น K-Nearest Neighbors แต่หากหาข้อมูลมาทดสอบไม่ได้ ก็สามารถใช้ฟังก์ชัน make_blobs() ของ Scikit-Leam เพื่อสร้างข้อมูลตัวอย่างมาใช้งานไปก่อน ซึ่งลักษณะของฟังก์ชัน make_blobs() และพารามิเตอร์ที่น่าสนใจคือ
- ตัวเลขในวงเล็บคือ ค่าดีฟอลต์

```
make_blobs (
n_samples=จำนวนรายการ (100),
n_features=จำนวนคอลัมน์ที่เป็น Feature (2),
centers=จำนวนศูนย์กลาง (3),
random_state=กำหนดเป็นจำนวนเต็มตั้งแต่ 0 ขึ้นไป เพื่อให้ได้ข้อมูลชุดเดิมตลอด
```


9. การสร้างข้อมูลตัวอย่างด้วย make_blobs (ต่อ)

- สำหรับแนวทางการใช้งาน จะวาดกราฟของความสัมพันธ์ระหว่างคอลัมน์ที่เป็น Feature ในแบบ Scatterให้ดูด้วย เพื่อให้เห็นลักษณะการแบ่งกลุ่ม

```
from sklearn.datasets import make_blobs
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
x, y = make_blobs(n_samples=100, centers=3, random_state=1)
df = pd.DataFrame(x, columns=['x1', 'x2'])
df['y'] = y
with pd.option_context('display.max_rows', 6): display(df)

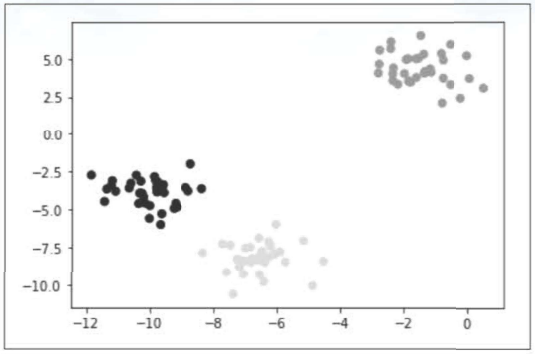
colors = np.array(['#999', '000', '#ddd'])
plt.scatter(df['x1'], df['x2'], color=colors[df['y']])
plt.show()
```



9. การสร้างข้อมูลตัวอย่างด้วย make_blobs (ต่อ)

	x1	x2	y
0	-0.794152	2.104951	0
1	-9.151552	-4.812864	1
2	-11.441826	-4.457814	1
...	...	...	...
97	-1.373973	5.291631	0
98	-6.253931	-7.108786	2
99	0.085252	3.645283	0

100 rows × 3 columns



10. การสร้างข้อมูลตัวอย่างด้วย make_circles

- ในบางโมเดล ข้อมูลที่จะนำมาทดสอบต้องมีลักษณะเป็นแนวโค้ง หรือคล้ายกับวงกลมหรือวงรี ดังนั้นการหาข้อมูลจริงมาทดสอบอาจยากนิดหน่อย เช่น โมเดล Support Vector Machines แต่ใน Scikit-Leam มีฟังก์ชัน make_circles() ที่สามารถสร้างข้อมูลเช่นนั้นได้ ซึ่งลักษณะพารามิเตอร์ที่น่าสนใจของฟังก์ชันดังกล่าวคือ
- ค่าในวงเล็บคือ ค่าดีฟอลต์

make_circles(
n_samples=จำนวนรายการ (100),
noise=ค่า Standard deviation สำหรับสร้างข้อมูล (None),
random_state=กำหนดเป็นจำนวนเต็มตั้งแต่ 0 ขึ้นไป เพื่อให้ได้ข้อมูลชุดเดิมตลอด

10. การสร้างข้อมูลตัวอย่างด้วย make_circles (ต่อ)

- สำหรับแนวทางการใช้งานจะวาดกราฟในแบบ Scatter ให้ดูด้วย เพื่อให้เห็นลักษณะการจัดเรียงตำแหน่งข้อมูล

```
from sklearn.datasets import make_circles
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

x, y = make_circles(n_samples=100, noise=0.08, random_state=0)

df = pd.DataFrame(x, columns=['x1', 'x2'])
df['y'] = y

with pd.option_context('display.max_rows', 6): display(df)

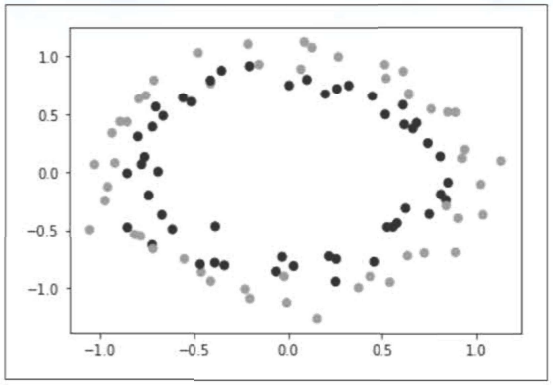
colors = np.array(['#999', '000'])
plt.scatter(df['x1'], df['x2'], color=colors[df['y']])
plt.show()
```



10. การสร้างข้อมูลตัวอย่างด้วย make_circles (ต่อ)

	x1	x2	y
0	-0.958083	-0.127571	0
1	-0.031826	-0.734087	1
2	0.939436	0.194388	0
...	...	...	...
97	0.003803	0.749047	1
98	0.842654	-0.289395	0
99	0.636232	-0.719470	0

100 rows × 3 columns



11. การตรวจสอบค่าที่สูญหาย (Missing Value)

- กรณีที่อ่านข้อมูลจากไฟล์หรือฐานข้อมูล อาจมีความเป็นไปได้ที่ข้อมูลบางส่วนจะขาดหายไป หรือ เรียกว่า Missing Value ดังในตารางต่อไปนี้

car	color	volume	weight	co2
Honda Civic	Red	1600	1252	94
Suzuki Swift	White	1300		101
Mazda 3	Black		1280	104
Benz CLA	Black		1465	102
Mini Cooper	Red	1500		105
...	...	...	...	...

11. การตรวจสอบค่าที่สูญหาย (Missing Value) (ต่อ)

- จากตารางจะเห็นได้ว่า ข้อมูลบางช่องในคอลัมน์ volume และ weight นั้นขาดหายไป ซึ่งหากนำไปใช้กับ Machine Learning โดยไม่มีการแก้ไข ผลลัพธ์ที่ได้อาจมีความคลาดเคลื่อนหรือบางกรณีก็อาจเกิดข้อผิดพลาดขึ้นได้ ดังนั้น หากเป็นการใช้ข้อมูลจากไฟล์หรือฐานข้อมูล ควรตรวจสอบก่อนว่ามีค่าว่างหรือไม่ ถ้ามีก็ต้องแก้ไขให้เรียบร้อยก่อนที่จะใช้งานในขั้นตอนต่อไป โดยหากแสดงผลด้วยไลบรารี Pandas ช่องเซลล์ที่ข้อมูลสูญหายจะแทนด้วยคำว่า NaN(Nota Number) ดังภาพ

	car	color	volume	weight	co2
0	Honda Civic	Red	1600.0	1252.0	94
1	Suzuki Swift	White	1300.0	NaN	101
2	Mazda 3	Black	NaN	1280.0	104
3	Benz CLA	Black	NaN	1465.0	102
4	Mini Cooper	Red	1500.0	NaN	105
5	Ford Focus	Blue	2000.0	1328.0	105
6	Benz C-Class	Silver	NaN	1365.0	99
7	Benz E-Class	While	2100.0	1605.0	115
8	Ford Fiesta	Red	1500.0	1112.0	98
9	Volvo XC70	Silver	2000.0	1746.0	117

11. การตรวจสอบค่าที่สูญหาย (Missing Value) (ต่อ)

- อย่างไรก็ตาม ไลบรารี Pandas ก็มีเมธอดสำหรับการตรวจสอบและแก้ไขข้อมูลที่สูญหาย ดังนี้

  - isnull().any() ตรวจสอบว่าในแต่ละคอลัมน์มีค่าที่เป็น NaN หรือไม่
  - isnull().sum() นับจำนวนรายการที่มีค่าเป็น NaN ในแต่ละคอลัมน์
  - dropna() ลบรายการ (แถว) ที่มีค่า NaN ทิ้งไป
  - fillna() เปลี่ยนข้อมูลในเซลล์ที่มีค่า NaN เป็นค่าที่ระบุ

11. การตรวจสอบค่าที่สูญหาย (Missing Value) (ต่อ)

- การดำเนินการกับค่าที่สูญหาย ควรทำกับคอลัมน์ที่จะนำไป Train Model ส่วนคอลัมน์ที่ไม่เกี่ยวข้อง ไม่จำเป็นต้องดำเนินการใดๆ เช่น สมมติว่า จัดเก็บข้อมูลไว้ในไฟล์แบบ CSV ซึ่งมีบางส่วนที่สูญหายไป ทั้งนี้เมื่อแสดงผลในแบบ DataFrame จะได้ดังภาพที่แล้ว หากต้องการตรวจสอบและจัดการกับค่า NaN ด้วย เมธอดของ DataFrame ดังที่สรุปในตารางที่ผ่านมา ก็มีแนวทางดังนี้

  - Isnull.any() เป็นการตรวจสอบว่า มีค่าที่สูญหายหรือไม่ หากในคอลัมน์ใดมีค่าที่สูญหายอย่างน้อย 1 เซลล์ จะแสดงคำว่า True แต่หากคอลัมน์ใดไม่มีค่าสูญหายเลย จะแสดงเป็นคำว่า False
  - isnull().sum() จะแสดงจำนวนค่าที่สูญหายทั้งหมดในแต่ละคอลัมน์

11. การตรวจสอบค่าที่สูญหาย (Missing Value) (ต่อ)

```
import pandas as pd
import numpy as np
df = pd.read_csv(r'dataltest-missing-values.csv')
#display (df)
print(df.isnull().any())
print()
print(df.isnull().sum())
#กรณีที่ต้องการตรวจสอบเพียงบางคอลัมน์ เช่น
#df['volume'].isnull().any()
#df[['volume', 'weight']].isnull().sum()
```

```
car       False
color     False
volume     True
weight     True
co2       False
dtype: bool
```

df.isnull().any() แสดงว่า 2 คอลัมน์นี้มีข้อมูลที่สูญหาย

```
car       0
color     0
volume    3
weight    2
co2       0
dtype: int64
```

df.isnull().sum() จำนวนข้อมูลที่สูญหายของ 2 คอลัมน์นี้

11. การตรวจสอบค่าที่สูญหาย (Missing Value) (ต่อ)

- dropna() ใช้สำหรับลบแถวที่มีค่าสูญหาย โดยข้อมูลจะถูกลบออกไปทั้งแถวในลักษณะดังนี้

```
import pandas as pd
import numpy as np
df = pd.read_csv(r'dataltest-missing-values.csv')
#display (df)
df.dropna(inplace=True) #inplace=True คือให้แทนที่ผลลัพธ์ในออบเจ็กต์เดิม
display (df)
```

	car	color	volume	weight	co2
0	Honda Civic	Red	1600.0	1252.0	94
1	Suzuki Swift	White	1300.0	NaN	101
2	Mazda 3	Black	NaN	1280.0	104
3	Benz CLA	Black	NaN	1465.0	102
4	Mini Cooper	Red	1500.0	NaN	105
5	Ford Focus	Blue	2000.0	1328.0	105
6	Benz C-Class	Silver	NaN	1365.0	99
7	Benz E-Class	While	2100.0	1605.0	115
8	Ford Fiesta	Red	1500.0	1112.0	98
9	Volvo XC70	Silver	2000.0	1746.0	117

	car	color	volume	weight	co2
0	Honda Civic	Red	1600.0	1252.0	94
5	Ford Focus	Blue	2000.0	1328.0	105
7	Benz E-Class	While	2100.0	1605.0	115
8	Ford Fiesta	Red	1500.0	1112.0	98
9	Volvo XC70	Silver	2000.0	1746.0	117

11. การตรวจสอบค่าที่สูญหาย (Missing Value) (ต่อ)

- fillna(..) คือการนำข้อมูลมาเติมแทนที่ค่าสูญหาย ซึ่งส่วนใหญ่นิยมใช้วิธีใดวิธีหนึ่งดังต่อไปนี้
- fillna(mean) คือการเติมค่าเฉลี่ยของคอลัมน์นั้น ซึ่งค่าดังกล่าวหาได้จากเมธอด mean() ของ DataFrame เช่น

11. การตรวจสอบค่าที่สูญหาย (Missing Value) (ต่อ)

```
import pandas as pd
import numpy as np

df = pd.read_csv(r'data\test-missing-values.csv')
#display(df)

v_mean = df['volume'].mean()
df['volume'].fillna(v_mean, inplace=True)

df['weight'].fillna(df['weight'].mean(), inplace=True)

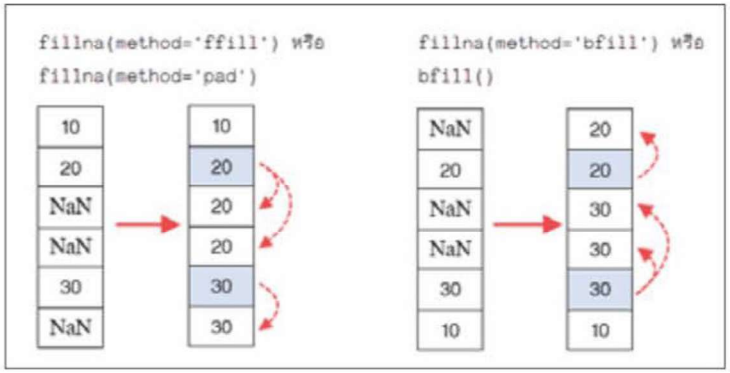
display(df)
```

	car	color	volume	weight	co2
0	Honda Civic	Red	1600.000000	1252.000	94
1	Suzuki Swift	White	1300.000000	1394.125	101
2	Mazda 3	Black	1714.285714	1280.000	104
3	Benz CLA	Black	1714.285714	1465.000	102
4	Mini Cooper	Red	1500.000000	1394.125	105
5	Ford Focus	Blue	2000.000000	1328.000	105
6	Benz C-Class	Silver	1714.285714	1365.000	99
7	Benz E-Class	While	2100.000000	1605.000	115
8	Ford Fiesta	Red	1500.000000	1112.000	98
9	Volvo XC70	Silver	2000.000000	1746.000	117



11. การตรวจสอบค่าที่สูญหาย (Missing Value) (ต่อ)

- fillna(method= ffill) หรือ fillna(method='pad') ซึ่งเรียกว่า fill value forward เป็นการนำค่าจากเซลล์ก่อนนี้ที่ไม่ใช่ NaN มาเติมลงในเซลล์ถัดไปที่มีค่า NaN
- fillna(method='bfil') หรือ bfill) ซึ่งเรียกว่า fll value backward เป็นการนำค่าค่าจากเซลล์ ถัดไปที่ไม่ใช่ NaN ย้อนกลับมาเติมลงในเซลล์ก่อนนี้ที่มีค่า NAN



11. การตรวจสอบค่าที่สูญหาย (Missing Value) (ต่อ)

```
import pandas as pd
import numpy as np
df = pd.read_csv(r'data\test-missing-values.csv')
#display(df)
df['volume'].fillna(method='ffill', inplace=True) #method='pad'
df['weight'].fillna(method='bfill', inplace=True)  #bfill()
display(df)
```

	car	color	volume	weight	co2
0	Honda Civic	Red	1600.0	1252.0	94
1	Suzuki Swift	White	1300.0	NaN	101
2	Mazda 3	Black	NaN	1280.0	104
3	Benz CLA	Black	NaN	1465.0	102
4	Mini Cooper	Red	1500.0	NaN	105
5	Ford Focus	Blue	2000.0	1328.0	105
6	Benz C-Class	Silver	NaN	1365.0	99
7	Benz E-Class	White	2100.0	1605.0	115
8	Ford Fiesta	Red	1500.0	1112.0	98
9	Volvo XC70	Silver	2000.0	1746.0	117

	car	color	volume	weight	co2
0	Honda Civic	Red	1600.0	1252.0	94
1	Suzuki Swift	White	1300.0	1280.0	101
2	Mazda 3	Black	1300.0	1280.0	104
3	Benz CLA	Black	1300.0	1465.0	102
4	Mini Cooper	Red	1500.0	1328.0	105
5	Ford Focus	Blue	2000.0	1328.0	105
6	Benz C-Class	Silver	2000.0	1365.0	99
7	Benz E-Class	White	2100.0	1605.0	115
8	Ford Fiesta	Red	1500.0	1112.0	98
9	Volvo XC70	Silver	2000.0	1746.0	117



12. การเข้ารหัสสตริงให้เป็นตัวเลข (Encoding)

- การประมวลผลของ Scikit-Leam ต้องใช้ข้อมูลที่เป็นตัวเลขเท่านั้น โดยเฉพาะอย่างยิ่งกับคอลัมน์ ที่เป็น Feature ส่วนคอลัมน์ที่เป็น Target หรือผลลัพธ์ ข้อมูลอาจเป็นสตริงก็ได้ (สำหรับการทำนายผลแบบ Classification) เช่น สมมติว่ามีข้อมูลดังตารางต่อไปนี้ หากนำไปใช้งานกับ Scikit-Learn จะเกิดข้อผิดพลาด

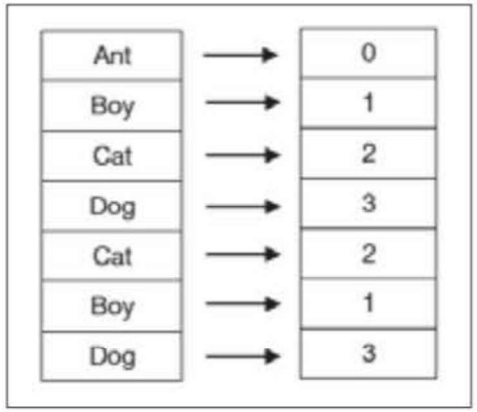
gender	income	purchased
Female	High	Yes
Female	Medium	No
Male	Low	No
Male	Medium	Yes
Female	Low	No

12. การเข้ารหัสสตริงให้เป็นตัวเลข (Encoding) (ต่อ)

- จากตาราง คอลัมน์ที่เป็น Feature คือ gender และ income ซึ่งข้อมูลเป็นสตริง ดังนั้น ก่อนจะนำไปใช้งานกับ Scikit-Learn ต้องเข้ารหัสหรือแปลงให้เป็นตัวเลขก่อน ส่วนคอลัมน์ purchased ซึ่งเป็น Target อาจจะเข้ารหัสหรือไม่ก็ได้ สำหรับการเข้ารหัสข้อมูลจากสตริงเป็นตัวเลขนั้น สามารถทำได้หลายวิธี โดยอาจใช้เมธอดการแทนที่ของ Pandas (DataFrame) ก็ได้ และใน Scikit-Learn ก็มีคลาสสำหรับกรณีนี้มาให้ใช้งานโดยตรงอยู่แล้ว เช่น LabelEncoder, OneHotEncoder เป็นต้น
- สำหรับ LabelEncoder มีรายละเอียดดังต่อไปนี้
  - คลาส LabelEncoder ใช้สำหรับการเข้ารหัสข้อมูลในแบบสตริงให้เป็นตัวเลขที่มีค่าระหว่าง 0 ถึง (จำนวนกลุ่ม - 1) เช่น

12. การเข้ารหัสสตริงให้เป็นตัวเลข (Encoding) (ต่อ)

- จากภาพ ข้อมูลแบ่งเป็น 4 กลุ่มคือ Ant. Boy, Cat และ Dog ดังนั้น หลังการเข้ารหัสจึงได้ตัวเลขระหว่าง 0-3 และข้อมูลที่มีค่าซ้ำกันจะถูกเข้ารหัสเป็นเลขตัวเดียวกัน แต่ต้องเขียนด้วยตัวพิมพ์ที่ตรงกันทุกอักขระ มิฉะนั้นจะถือว่าเป็นข้อมูลคนละกลุ่ม



12. การเข้ารหัสสตริงให้เป็นตัวเลข (Encoding) (ต่อ)

- สำหรับแนวทางการเข้ารหัสสตริงเป็นตัวเลขโดยใช้ LabelEncoder มีดังนี้
- คลาส LebelEncoder อยู่ในโมดูล sklearn.preprocessing ซึ่งต้องอิมพอร์ตก่อนการใช้สร้างอินสแตนซ์ของมันในลักษณะดังนี้

```
rom sklearn.preprocessing import LabelEncder
…
encoder = LabelEncoder()
```

- ปกติแล้วข้อมูลที่จะเข้ารหัสด้วย LabelEncoder ต้องเป็นแบบลิสต์หรืออาร์เรย์ กรณีของ DataFrameจะเข้ารหัสได้ครั้งละ 1 คอลัมน์ ยกเว้นจะใช้เทคนิคพิเศษที่จะกล่าวถึงในภายหลัง

12. การเข้ารหัสสตริงให้เป็นตัวเลข (Encoding) (ต่อ)

- การเข้ารหัสข้อมูลแบบที่ 1 มีแนวทางดังนี้
  - เรียกเมธอด fit(data) เพื่อสร้างรหัสอ้างอิงหรือหลักเกณฑ์ในการเข้ารหัส (แต่ละคำเป็นเลขอะไรนั่นเอง)
  - เรียกเมธอด transform(data) เพื่อทำการเข้ารหัสข้อมูล
  - ข้อมูลแต่ละค่าที่จะเข้ารหัสด้วย transform() ต้องเคยเข้ารหัสด้วยเมธอด ft() มาแล้ว นอกจากนี้ยังต้องเขียนด้วยตัวพิมพ์แบบเดียวกันทุกอักขระ
  - ถ้าจะเข้ารหัสข้อมูลที่เป็นชุดเดียวกันครั้งต่อ ๆ ไป ให้เรียกเมธอด transform เพื่อเข้ารหัสได้เลยอย่าเรียกเมธอด ft() ซ้ำอีก เพราะจะเป็นการสร้างรหัสอ้างอิงขึ้นมาใหม่ ซึ่งอาจไม่สอดคล้องกับชุดเดิม ยกเว้นจะเป็นข้อมูลคนละชุด
  - สรุปได้ว่า จะเรียก ft() เพียงครั้งเดียว แล้วต่อๆ ไปก็เรียก transform() ได้เลย

12. การเข้ารหัสสตริงให้เป็นตัวเลข (Encoding) (ต่อ)

```
from sklearn.preprocessing import LabelEncoder
encoder = LabelEncoder()
data = ['Apple', 'Mango', 'Grape', 'Durian', 'Mango', 'Grape']
encoder.fit(data)
enc_data1 = encoder.transform(data)
print(enc_data1)
# [0 3 2 1 3 2]
#ถ้าจะเข้ารหัสข้อมูลครั้งต่อไป (ข้อมูลชุดเดียวกัน) ให้เรียกเมธอด transform ) ได้เลย
enc_data2 = encoder.transform(['Mango', 'Durian', 'Durian'])
print(enc_data2)
#[3 1 1]
```

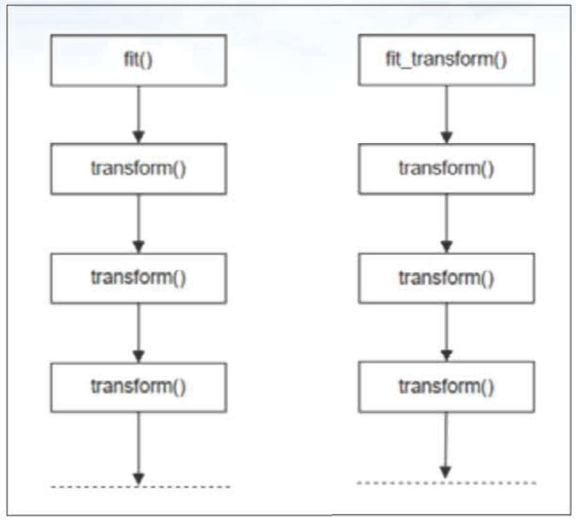
12. การเข้ารหัสสตริงให้เป็นตัวเลข (Encoding) (ต่อ)

```
enc_data3 = encoder.transform(['Grape', 'Mango', 'Mango', 'Durian'])
print (enc_data3)
#[2 3 3 1]
enc data4 = encoder.transform(['mango', 'Mangosteen', 'Melon']) #Error
#เกิดข้อผิดพลาด เพราะ
#mango เขียนด้วยตัวพิมพ์ที่ไม่ตรงกับกับข้อมูลที่กำหนดให้แก่ 61t ()
#Mangosteen และ Melon  ไม่อยู่ในชุดข้อมูลที่กำหนดให้แก่i fit)
```


12. การเข้ารหัสสตริงให้เป็นตัวเลข (Encoding) (ต่อ)

- การเข้ารหัสข้อมูลแบบที่ 2 มีแนวทางดังนี้
  - เรียกเมธอด fit_tranform(data) เพื่อสร้างรหัสอ้างอิงหรือหลักเกณฑ์ในเข้ารหัส พร้อมกับเข้ารหัสข้อมูลต่อไปทันที หรือเป็นการรวม ft() และ tranform() ไว้ในขั้นตอนเดียวกันนั่นเอง
  - ถ้าจะเข้ารหัสข้อมูลที่เป็นชุดเดียวกันครั้งต่อ ๆ ไป ให้เรียก เมธอด transform() เพื่อเข้ารหัสได้เลยอย่าเรียกเมธอด fit_transform() ซ้ำอีก เพราะจะเป็นการสร้างรหัสอ้างอิงขึ้นมาใหม่ ซึ่งอาจไม่สอดคล้องกับชุดเดิม ยกเว้นจะเป็นข้อมูลคนละชุด
  - สรุปได้ว่า จะเรียก ft_transform() เพียงครั้งเดียว แล้วต่อๆ ไปก็เรียก transform() ได้เลย (สำหรับข้อมูลชุดเดียวกัน)
  - หลักการปลีกย่อยอื่นๆ ก็คล้ายกับ ft()

12. การเข้ารหัสสตริงให้เป็นตัวเลข (Encoding) (ต่อ)



12. การเข้ารหัสสตริงให้เป็นตัวเลข (Encoding) (ต่อ)

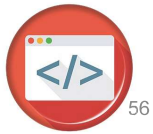
```
from sklearn.preprocessing import LabelEncoder

data = ['Apple', 'Mango', 'Grape', 'Durian', 'Mango', 'Grape']
encoder = LabelEncoder()

enc_data1 = encoder.fit_transform(data)
print(enc_data1)

enc_data2 = encoder.transform(['Mango', 'Durian', 'Durian'])
print(enc_data2)

enc_data2 = encoder.transform(['Apple', 'Grape', 'Durian', 'Grape'])
print(enc_data2)
```



12. การเข้ารหัสสตริงให้เป็นตัวเลข (Encoding) (ต่อ)

- ผลจากการใช้เมธอด tranform() หรือ fit_transform() จะได้ค่าที่เป็นตัวเลข หากต้องการถอดรหัสข้อมูล หรือแปลงกลับจากตัวเลขไปเป็นสตริง ให้เรียกเมธอด inverse_transform() พร้อค่าที่เป็นตัวเลขที่อยู่ช่วงของการเข้าข้อมูดนั้น (0 ถึง ถึงจำนวนกลุ่ม -1) เช่น

12. การเข้ารหัสสตริงให้เป็นตัวเลข (Encoding) (ต่อ)

```
from sklearn.preprocessing import LabelEncoder

data = ['Apple', 'Mango', 'Grape', 'Durian', 'Mango', 'Grape']
encoder = LabelEncoder()

enc_data = encoder.fit_transform(data)
print(enc_data)

#แปลงจากตัวเลขกลับไปเป็นสตริง
inv = encoder.inverse_transform(enc_data)
print(inv)

print(encoder.inverse_transform([1, 1, 0, 2]))

#print(encoder.inverse_transform([1, 0, 4, 5])) #Error
#เกิดข้อผิดพลาด เพราะ 4 และ 5 ไม่อยู่ในขอบเขตของเลขที่แปลงได้ของข้อมูลชุดนี้
```



12. การเข้ารหัสสตริงให้เป็นตัวเลข (Encoding) (ต่อ)

- กรณีของ DataFranme จะเข้ารหัสได้ครั้งละ 1 คอลัมน์เท่านั้น หากต้องการเข้ารหัสมากกว่า 1 คอลัมน์ต้องสร้าง Encoder แยกกัน เช่น สมมติว่าจัดเก็บข้อมูลไว้ในไฟล์แบบ Excel ซึ่งในแต่ละคอลัมน์จะเป็นสตริง หากต้องการเข้ารหัสเป็นตัวเลขก็อาจทำดังนี้

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder
df = pd.read_excel(r'dataltest-label-encoder.xlsx')
display (df)

enc_weather = LabelEncoder()
enc_weather.fit(df['Weather'])
df['Weather'] = enc_weather.transform(df['Weather'])
```


12. การเข้ารหัสสตริงให้เป็นตัวเลข (Encoding) (ต่อ)

```
enc_timeofweek = LabelEncoder().fit(df['TimeOfweek'])
df['TimeOfweek'] = enc_timeofweek.transform(df['TimeOfweek']]]])
df['TimeOfDay'] = LabelEncoder().fit_transform(df["TimeOfDay'])
display (df)
#คอลัมน์ TrafficJam ไม่ได้เข้ารหัส จึงเป็นสตริงเหมือนเดิม
```

	Weather	TimeOfWeek	TimeOfDay	TrafficJam
0	Clear	Workday	Morning	Yes
1	Clear	Workday	Evening	Yes
2	Clear	Weekend	Lunch	No
3	Rainy	Workday	Morning	Yes
4	Rainy	Workday	Lunch	Yes
5	Rainy	Workday	Evening	Yes
6	Rainy	Weekend	Lunch	No
7	Snowy	Workday	Morning	Yes
8	Snowy	Workday	Evening	Yes
9	Snowy	Weekend	Lunch	No

	Weather	TimeOfWeek	TimeOfDay	TrafficJam
0	0	1	2	Yes
1	0	1	0	Yes
2	0	0	1	No
3	1	1	2	Yes
4	1	1	1	Yes
5	1	1	0	Yes
6	1	0	1	No
7	2	1	2	Yes
8	2	1	0	Yes
9	2	0	1	No

12. การเข้ารหัสสตริงให้เป็นตัวเลข (Encoding) (ต่อ)

- สามารถเข้ารหัสข้อมูลของ DataFrame หลายคอลัมน์พร้อมกันได้ ซึ่งสามารถทำได้หลายวิธีแต่ที่น่าจะง่ายที่สุดคือการใช้เมธอด apply() ของ DataFrame แล้วกำหนดเมธอด fit_tranform()
- ถ้าต้องการเข้ารหัสทุกคอลัมน์

```
import pandas as pd
import numpy as np
rom sklearn.preprocessing import LabelEr
' = pd.read_excel(r'dataltest-label-encoder.x
#display (df)
encoder = LabelEncoder()
#ถ้าต้องการเข้ารหัสทุกคอลัมน์
df = df.apply(encoder.fit_transform)
print(df)
```


12. การเข้ารหัสสตริงให้เป็นตัวเลข (Encoding) (ต่อ)

- ถ้าต้องการเข้ารหัสแบบเจาะจงคอลัมน์

```
...
encoder = LabelEncoder()
#ถ้าต้องการเข้ารหัสเฉพาะคอลัมน์ Weather
f['Weather'] = df['Weather'].apply(encoder.fit_tra
#ถ้าต้องการเข้ารหัสหลายคอลัมน์
cols = ['Weather', 'TimeOfWeek', 'TimeOfDay']
ols] = df[cols].apply(encoder.fit_trans
#หรือวิธีอื่นๆ (เข้ารหัสคอลัมน์ลำดับที่ 0 - 2)
Df.iloc[:, 0:3] = df.iloc[:, 0:3].apply(encoder.fit_transform)
```


13. การปรับสเกลของข้อมูล

- ข้อมูลที่จะนำมา Train Model นั้น หากบางคอลัมน์มีค่าแตกต่างจากคอลัมน์อื่นๆ มากเกินไป อาจให้คอลัมน์นั้นมีน้ำหนักมากกว่า หรือไม่สมดุลกับข้อมูลในคอลัมน์อื่นๆ ซึ่งอาจส่งผลต่อการทำนาย และเกิดความคลาดเคลื่อนได้ เช่น สมมติว่าข้อมูลตัวอย่าง เป็นดังนี้

car	color	volume	weight	co2
Honda Civic	Red	1.6	1252	94
Suzuki Swift	White	1.3	990	101
Mazda 3	Black	2.2	1280	104
Benz CLA	Black	1.5	1465	102
Mini Cooper	Red	1.5	1140	105
Ford Focus	Blue	2.0	1328	105
Benz C-Class	Silver	2.1	1365	99
Benz E-Class	While	2.1	1605	115
Ford Fiesta	Red	1.5	1112	98
Volvo XC70	Silver	2.0	1746	117

13. การปรับสเกลของข้อมูล (ต่อ)

- จากตาราง ถ้า volume และ weight คือคอลัมน์ที่เป็น Feature จะเห็นได้ว่า คอลัมน์ volume มีค่าในหลักหน่วย แต่คอลัมน์ weight มีค่าในหลักพัน
- ดังนั้นคอลัมน์ weight มีจึงน้ำหนักต่อการทำนายผลมากกว่าคอลัมน์ volume อย่างชัดเจน ซึ่งหากข้อมูลมีลักษณะเช่นนี้ ควรแก้ไขด้วยการปรับสเกลใหม่ให้ใกล้เคียงกันมากขึ้น โดย Scikit-Leam มีคลาสที่ใช้ในการปรับสเกลของข้อมูลหลายอัน เช่น StandardScaler. MaxScaler, MaxAbScaler, RobustScaler เป็นต้น แต่โดยส่วนใหญ่นิยมใช้ Standar ที่สุด

13. การปรับสเกลของข้อมูล (ต่อ)

- คลาส StandardScaler จะคำนวณโดยใช้สูตร

$$z = \frac{x-u}{s}$$

z คือค่าใหม่หลังการปรับสเกล

x คือค่าเดิม หรือข้อมูลตัวอย่างแต่ละค่า

u คือค่าเฉลี่ยของข้อมูลในคอลัมน์นั้น

s คือส่วนเบี่ยงเบนมาตรฐานของข้อมูลในคอลัมน์นั้น

13. การปรับสเกลของข้อมูล (ต่อ)

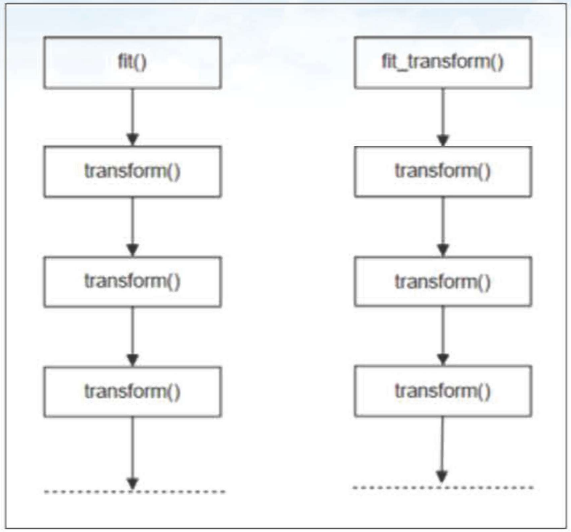
- อย่างไรก็ตาม ไม่จำเป็นต้องคำนวณเอง เพราะใน Scikit-Learn จะดำเนินการให้กับทั้งหมด ดังขั้นตอนต่อไปนี้
- คลาส StandardScaler อยู่ในโมดูล skleam.preprocessing ซึ่งต้องอิมพอร์ตก่อนการใช้งาน แล้วสร้างอินสแตนซ์ในลักษณะดังนี้

```
from sklearn.preprocessing import StandardScaler
...
scaler = StandardScaler()
```


13. การปรับสเกลของข้อมูล (ต่อ)

- การปรับสเกลจะมีหลักการคล้ายกับ LabelEncoder ดังนี้
  - วิธีที่ 1 เรียกเมธอด fidata) เพื่อสร้างสเกลอ้างอิง แล้วตามด้วยเมธอด transform(data)เพื่อทำการปรับสเกลข้อมูล
  - วิธีที่ 2 เรียกเมธอด fit_transform(data) เพื่อทำ 2 ขั้นตอนต่อเนื่องกัน
  - ถ้าจะปรับสเกลข้อมูลที่เป็นชุดเดียวกันครั้งต่อ ๆ ไปให้เรียกเมธอด transform() เพื่อปรับสเกลได้เลยอย่าเรียกเมธอด fit() หรือ fit_transform() ซ้ำอีก เพราะจะเป็นการสร้างสเกลอ้างอิงขึ้นมาใหม่ซึ่งอาจไม่สอดคล้องกับชุดเดิม ยกเว้นจะเป็นข้อมูลคนละชุด หรือสรุปได้ว่า จะเรียก fit() หรือ fit_transforml) เพียงครั้งเดียว แล้วต่อๆ ไปก็เรียก tansform() ได้เลย

13. การปรับสเกลของข้อมูล (ต่อ)



13. การปรับสเกลของข้อมูล (ต่อ)

- StandardScaler สามารถปรับสเกลพร้อมกันได้มากกว่า 1 คอลัมน์ ซึ่งตรงนี้จะต่างจาก LabelEncoder ที่ทำได้ทีละคอลัมน์เท่านั้น สำหรับแนวทางการใช้งานในเบื้องต้นมีดังนี้

```
from sklearn.preprocessing import StandardScaler
data = [[2, 2000],
        [3.5, 5500],
        [1.75, 2150],
        [3.14, 4130],
        [2.25, 2564]]
scaler = StandardScaler()
scaler.fit(data)
data2 = scaler.transform(data)
print(data2)
print()
data_test = [[1.23, 3210]]
print(scaler.transform(data_test))
```

```
[[-0.78178395 -0.9416242 ]
 [ 1.43919317  1.65585743]
 [-1.1519468  -0.83030356]
 [ 0.90615866  0.63912891]
 [-0.41162109 -0.52305859]]

[[-1.92188553 -0.04363769]]
```



13. การปรับสเกลของข้อมูล (ต่อ)

- ส่วนแนวทางการใช้ร่วมกับ DataFrame เช่น สมมติว่าเก็บข้อมูลเกี่ยวกับการปล่อยก๊าซ CO2 ไว้ในไฟล์แบบ CSV หากต้องการปรับสเกลของคอลัมน์ volum ก็ทำดังนี้

```
from sklearn.preprocessing import StandardScaler
import pandas as pd
import numpy as np
scaler = StandardScaler()
df = pd.read_csv(r'dataltest-standard-scaler.csv')
#display (df)
#ถ้าใช้วิธีเรียก fit() และตามด้วย transform()
scaler.fit(df[['volume', 'weight']])
df[['volume', 'weight']] = scaler.transform(df[[['volume', 'weight ]]]]
display (df)
#ถ้าใช้วิธีเรียก fit_transform() ในขั้นตอนเดียว
df[['volume', 'weight']] = scaler.fit_transform(df[['volume', 'weight ]]])
display ( df)
```


13. การปรับสเกลของข้อมูล (ต่อ)

```
#ถ้าต้องการปรับสเกลข้อมูลเพิ่มเติม
#ต้องกำหนดในแบบลิสต์หรืออาร์เรย์ 2 มิติ
ละต้องมีจำนวนคอลัมน์เท่ากับที่กำหนดให้แก่ fit() หรือ fit_transform
data = [[1.9, 1000]]
print(scaler.transform(data))
print(scaler.transform([[[2.3, 1234]]))
```

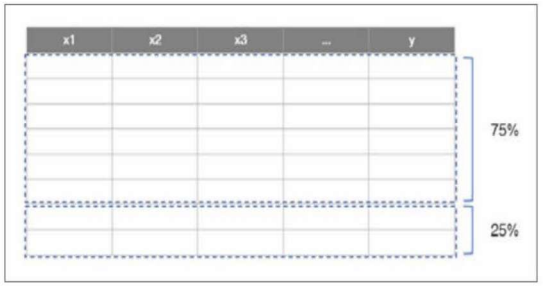
	car	color	volume	weight	co2
0	Honda Civic	Red	1.6	1252	94
1	Suzuki Swift	White	1.3	990	101
2	Mazda 3	Black	2.2	1280	104
3	Benz CLA	Black	1.5	1465	102
4	Mini Cooper	Red	1.5	1140	105
5	Ford Focus	Blue	2.0	1328	105
6	Benz C-Class	Silver	2.1	1365	99
7	Benz E-Class	While	2.1	1605	115
8	Ford Fiesta	Red	1.5	1112	98
9	Volvo XC70	Silver	2.0	1746	117

	car	color	volume	weight	co2
0	Honda Civic	Red	-0.576166	-0.349752	94
1	Suzuki Swift	White	-1.536443	-1.550735	101
2	Mazda 3	Black	1.344387	-0.221403	104
3	Benz CLA	Black	-0.896258	0.626620	102
4	Mini Cooper	Red	-0.896258	-0.883150	105
5	Ford Focus	Blue	0.704203	-0.001375	105
6	Benz C-Class	Silver	1.024295	0.168229	99
7	Benz E-Class	While	1.024295	1.268367	115
8	Ford Fiesta	Red	-0.896258	-0.991499	98
9	Volvo XC70	Silver	0.704203	1.914698	117

```
[[ 0.38411064 -1.50489638]]
[[ 1.66447944 -0.43226235]]
```

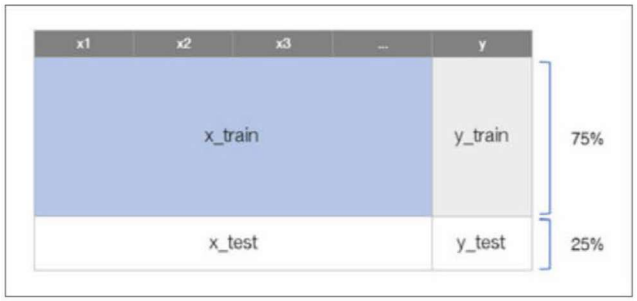

14. การแบ่งข้อมูลเป็นชุด Train และ Test

- หากข้อมูลตัวอย่างที่นำมาใช้ในการ Train Model มีจำนวนมากเกินไป ก็อาจทำให้เกิดความล่าช้าในการประมวลผล เช่น ข้อมูลบางชุดอาจมีจำนวนแถวเป็นหมื่นหรือเป็นแสนหรือมากกว่านั้น ซึ่งในกรณีอาจแบ่งข้อมูลออกเป็น 2 ชุด คือ
  - Training Set เป็นชุดข้อมูลที่ใช้ในการ Train Model เช่น ซึ่งตามปกติมักจะเลือก 75% ของข้อมูลทั้งหมดเพื่อเป็นชุดสำหรับการ Training
  - Testing Set เป็นชุดข้อมูลที่ใช้ในการทดสอบหรือการคำนวณความถูกต้อง ซึ่งตามปกติมักจะเลือก 25% ของข้อมูลทั้งหมดเพื่อเป็นชุดสำหรับ Testing



14. การแบ่งข้อมูลเป็นชุด Train และ Test (ต่อ)

- การแบ่งชุดข้อมูลในลักษณะดังกล่าว สามารถทำได้ง่ายๆ โดยใช้ฟังก์ชัน train_test_spit() ของโลบรารี Solkt-leam ซึ่งผลลัพธ์ที่ได้จากฟังก์ชันนี้ จะแบ่งข้อมูลออกเป็น 4 กลุ่มคือ
  - x_train คือชุดข้อมูลของคณลัมน์ที่เป็น Feature ซึ่งจะใช้ในการ Train Model
  - y_train คือชุดข้อมูลของคอลัมน์ที่เป็นผลลัพธ์ ซึ่งจะใช้ในการ Train Model ร่วมกับ x_train
  - x_test คือชุดข้อมูลของคอลัมน์ที่เป็น Feature ซึ่งจะใช้ใช้ในการทดสอบและประเมินคว
  - y_est คือชุดข้อมูลของคอลัมน์ที่เป็นผลลัพธ์ ซึ่งจะใช้ในการทดสอบและประเมินความถูกต้องร่วมกับ x_test



14. การแบ่งข้อมูลเป็นชุด Train และ Test (ต่อ)

- ฟังก์ชันที่ใช้ในการแบ่งชุดข้อมูลดังกล่าวคือ train_test_spit() ซึ่งอยู่ในโมดูล sklearn.model_selection โดยมีพารามิเตอร์ที่น่าสนใจดังนี้

```
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size, random_state)
```

- x_train, x_test, y_train, y_test คือผลลัพธ์ในแต่ละกลุ่มที่ได้จากฟังก์ชัน train_test_split) ซึ่งสามารถนำไปใช้งานได้ทันที
- x. y คือข้อมูลที่จะใช้ในการแบ่ง ซึ่งอาจอยู่ในแบบลิสต์/อาร์เรย์ หรือ DataFrame
- test_size เป็นการระบุสัดส่วนข้อมูลในการทดสอบ (Testing Set) โดยกำหนดเป็นตัวเลขระหว่าง0 - 1 เช่น 0.3 หมายถึง สัดส่วนข้อมูลทดสอบคือ 30% ซึ่งค่าดีฟอลต์คือ 0.25 (25%) และข้อมูลมูลที่เหลือถือเป็น Training Set โดยอัตโนมัติ

14. การแบ่งข้อมูลเป็นชุด Train และ Test (ต่อ)

- random_state อาจกำหนดเป็น 0 หรือตัวเลขอื่นๆ ก็ได้ เพื่อใช้สร้างสถานะของการสุ่มเลือกข้อมูลทั้งนี้หากกำหนดเป็นเลขตัวเดิมก็จะได้ข้อมูลชุดเดิมไปตลอด แต่ถ้าเปลี่ยนค่า random_state รายการที่ถูกเลือกเป็น Training Set/Testing Set จะแตกต่างออกไป ซึ่งต้องทดลองเปลี่ยนค่า random_state หลายๆ ค่า เพื่อหาตัวเลขที่จะให้ผลการทำนายมีความถูกต้องมากที่สุด

14. การแบ่งข้อมูลเป็นชุด Train และ Test (ต่อ)

- สำหรับวิธีการใช้งานในเบื้องต้น ตัวอย่างต่อไปนี้

```
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
#สร้างชุดข้อมูลตัวอย่างด้วย make_classification() ดังที่ได้ศึกษามาแล้ว

x, y = make_classification(n_samples=10, n_features=4,
n_classes=2, random_state=0)
x_train, x_test, y_train, y_test =
train_test_split(x, y, random_state=0)
print(x_train)
print(y_train)
print()
print(x_test)
print(y_test)
```

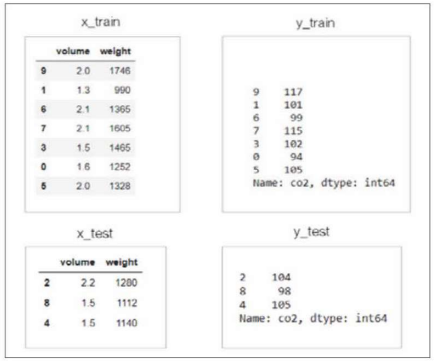
```
[[-0.0568887   1.20960511 -0.98740462  0.99958558]
 [-0.04621249  1.48755048 -1.19536732  1.27804656]
 [-0.14778398 -0.52978546  0.29484027 -0.79249401]
 [-0.7187573   2.14171734 -2.24080141  0.50054121]
 [-0.66261566  2.37124104 -2.3748206   0.82768797]
 [-0.01669425 -0.84862223  0.6476239  -0.81753611]
 [ 1.46552373 -1.56101351  2.38363567  1.56914464]]   x_train
[1 1 0 0 0 1]                                         y_train

[[-0.80328386  1.23271508 -1.60020786 -0.51203824]
 [ 1.00732222 -0.91817318  1.51783379  1.22140561]
 [-0.06642482  1.77341402 -1.43411205  1.50037656]]   x_test
[0 1 1]                                               y_test
```

14. การแบ่งข้อมูลเป็นชุด Train และ Test (ต่อ)

- แนวทางการแบ่งข้อมูลที่อยู่ในรูปแบบของ DataFrame เช่น จากไฟล์ตัวอย่างที่ใช้ทดสอบในหัวข้อการปรับสเกลข้อมูล จะอ่านเข้ามาเก็บใน DataFrame แล้วแบ่งเป็นชุด Training/Testing ดังนี้

```
from sklearn.model_selection import train_test_split
import pandas as pd
df = pd.read_csv(r'dataltest-standard-scaler.csv')
x = df.iloc[: 2:4] #df[['volume', 'weight']]
y = df['co2']
x_train, x_test, y_train, y_test =
train_test_split(x, y, random_state=0)
display (x_train)
display (y_train)
print()
display (x_test)
display ( y_test)
```



14. การแบ่งข้อมูลเป็นชุด Train และ Test (ต่อ)

- การกำหนดค่า random_state ด้วยตัวเลขที่ต่างกัน รายการข้อมูลที่ถูกคัดเลือกก็จะต่างกันออกไปด้วย เช่น Code และผลลัพธ์ต่อไปนี้ให้ลองเปรียบเทียบรายการในกลุ่ม x_train ที่ถูกคัดเลือกเมื่อกำหนดค่า random _state ที่แตกต่างกัน

```
from sklearn.model_selection import train_test_split
import pandas as pd
df = pd.read_csv(r'dataltest-standard-scaler.csv')
x = df[['volume', 'weight']]
y = df[ ' co2']
x_train,__,_, _ = train_test_split(x, y, random_state=0)
display ( x_train)
print('random_state = 0')
```


14. การแบ่งข้อมูลเป็นชุด Train และ Test (ต่อ)

```
x_train, _, _, = train_test_split(x, y, random_state=10)
display (x_train)
print('random_state = 10')
x_train, _, _, = train_test_split(x, y, random_state=100)
display ( x_train)
print( ' random_state = 100')
```

	volume	weight
9	2.0	1746
1	1.3	990
6	2.1	1365
7	2.1	1605
3	1.5	1465
0	1.6	1252
5	2.0	1328

random_state = 0

	volume	weight
6	2.1	1365
3	1.5	1465
1	1.3	990
0	1.6	1252
7	2.1	1605
4	1.5	1140
9	2.0	1746

random_state = 10

	volume	weight
5	2.0	1328
4	1.5	1140
2	2.2	1280
0	1.6	1252
3	1.5	1465
9	2.0	1746
8	1.5	1112

random_state = 100

14. การแบ่งข้อมูลเป็นชุด Train และ Test (ต่อ)

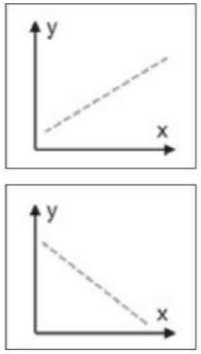
- กรณีที่มีทั้งการปรับสเกลข้อมูลและแบ่งเป็นชุด Training/Testing ควรแบ่งข้อมูลด้วย train_test_ spiti() ก่อน แล้วค่อยนำ x_train และ x_test ไปปรับสเกล เช่น

```
...
x_train, x_test, y_train, y_test = train_test_split(...)
#scaling after splitting
scaler = StandardScaler()
x_train = scaler.fit_transform(x_train)
x_test = scaler.transform(t_test)
...
```

- หลักการดังกล่าวนี้เรียกว่า Scaling After Spiting

15. ความสัมพันธ์ระหว่างคอลัมน์

- ข้อมูลที่จะนำมาใช้กับโมเดลบางชนิด ค่าของคอลัมน์ที่เป็น Feature หรือตัวแปรอิสระ (x) จะต้องมีผลให้คอลัมน์ที่เป็น Target หรือตัวแปรตาม (y) เกิดการเปลี่ยนแปลงหรือมีความสัมพันธ์แบบใดแบบหนึ่งดังนี้
  - ความสัมพันธ์แบบ Positive หมายถึง เมื่อค่าของตัวแปรอิสระเปลี่ยนแปลงไปค่าของตัวแปรตาม (y) จะเปลี่ยนแปลงในทิศทางเดียวกัน เช่น เพิ่มเหมือนกัน หรือลดเหมือนกัน เป็นต้น แม้จะไม่สม่ำเสมอก็ตาม
  - ความสัมพันธ์แบบ Negative หมายถึง เมื่อค่าของตัวแปรอิสระเปลี่ยนแปลงไปค่าของตัวแปรตาม (y) จะเปลี่ยนแปลงในทิศทางตรงกันข้าม เช่น ถ้าตัวแปรอิสระเพิ่ม ค่าของตัวแปรตามจะลดลง เป็นต้น



15. ความสัมพันธ์ระหว่างคอลัมน์ (ต่อ)

- แม้จะสามารถตรวจสอบความสัมพันธ์ระหว่างตัวแปรได้หลายวิธี เช่น การวาดกราฟแบบ Linear หรือกราฟแบบ Heatmap ของ Seabom แต่รูปแบบพื้นฐานที่จะแนะนำให้รู้จักเอาไว้ล่วงหน้า ก็คือการใช้เมธอด df.corr() ของ DataFrame โดยจะได้ผลลัพธ์เป็นตัวเลขมีค่าอยู่ระหว่าง -1.0 ถึง 1.0 ซึ่งแสดงความสัมพันธ์ระหว่างคอลัมน์ที่เป็นตัวเลข และมีความหมายดังนี้
  - มีค่าเข้าใกล้ 0 แสดงว่ามีความสัมพันธ์กันในระดับต่ำ นั่นคือ ข้อมูลในคอลัมล์มน์หนึ่งไม่ส่งผลให้อีกคอลัมน์หนึ่งเกิดการเปลี่ยนแปลงหรือเปลี่ยนน้อยมาก หรืออาจเรียกว่า แทบจะ ไม่มีผลต่อกัน
  - มีค่าเป็นบวก เช่น 0.85 แสดงว่าเป็นความสัมพันธ์แบบ Positive ดังที่กล่าวมาแล้ว และยิ่งตัวเลขมีค่ามากก็ยิ่งมีความสัมพันธ์ในเชิงบวกต่อกันมาก
  - มีค่าเป็นลบ เช่น -0.95 แสดงว่าเป็นความสัมพันธ์แบบ Negative ยิ่งตัวเลขที่ติดลบมีค่ามากก็ยิ่งมีความสัมพันธ์ในเชิงลบต่อกันมาก

15. ความสัมพันธ์ระหว่างคอลัมน์ (ต่อ)

- เช่น จากไฟล์ข้อมูลเกี่ยวกับปริมาณการปล่อยก๊าซคาร์บอนไดออกไซด์ที่ใช้ในตัวอย่างก่อนๆ นี้ หากสามารถตรวจสอบระดับความสัมพันธ์ระหว่างข้อมูลในแต่ละคอลัมน์จะได้ดังนี้

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

df = pd.read_csv(r'data\test-standard-scaler.csv')

display(df.corr().round(2))
```

	volume	weight	co2
volume	1.00	0.62	0.48
weight	0.62	1.00	0.73
co2	0.48	0.73	1.00

- ระดับความสัมพันธ์ของคอลัมน์ ให้ดูจากช่องเซลล์ที่เป็นจุดตัดในแนวนอนและแนวตั้งของคอลัมน์นั้น เช่น
  - คอลัมน์ volume และ co2 มีระดับความสัมพันธ์เป็น 0.48
  - คอลัมน์ weight และ co2 มีระดับความสัมพันธ์เป็น 0.73
  - คอลัมน์เดียวกัน ระดับความสัมพันธ์จะเป็น 1

